Assignment 2: DEEP LEARNING

Instructor: Prof. O. Olawale Awe

Instructions

- This assignment must be completed individually.
- Use a **real time series or image dataset** (e.g., Jena Climate Dataset, precipitation data, temperature series, energy consumption series).
- Clearly label all plots, tables, and equations.
- Cite all external sources.
- Use of OpenAI tools is prohibited.

Submission Checklist

- PDF report (4–10 pages)
- Clean and executable code (Python, PyTorch, or TensorFlow/Keras)
- Figures, tables, and interpretations
- Dataset source included

1 Dataset and Exploratory Analysis

You are required to work with a **real multivariate or univariate time series dataset**. Examples include:

- Jena Climate Dataset (temperature, pressure, humidity, etc.)
- West African precipitation or temperature time series
- Electricity consumption dataset
- Traffic volume dataset

Tasks

1. Load and clean the dataset. Handle missing values appropriately.
2. Select one target variable for prediction.
3. Produce a time-series plot and comment on trends, seasonality, and noise.
4. Perform chronological train/validation/test split and justify your method.

Figure 1: Time series plot of selected variable.

Commentary:

2 Sequence Construction and Normalisation

Create sliding-window sequences for two prediction tasks:

- **Use (Many-to-One):** Input = past 72 time steps, Output = value at $t + 24$.

Tasks

1. Implement a function to transform raw data into input–output windows.
2. Normalise features using **training statistics only**.
3. Provide code snippet and shape checks.

Equation for Window Construction:

$$X_t = (x_{t-71}, \ldots, x_t), \qquad y_t = x_{t+24}$$

Shape Summary:

3 Model 1: Baseline Feedforward Neural Network

Train a simple FFNN on Task A for comparison.

Tasks

1. Construct a Dense model using flattened windows.
2. Evaluate using MAE and RMSE (validation and test).

Results Table:

Model	MAE	RMSE
FFNN Baseline		

4 Model 2: Vanilla RNN

Implement a Vanilla RNN for Task A.

Tasks

1. Train a Vanilla RNN with at least 32 units.
2. Provide learning curves and evaluation metrics.
3. Explain the RNN update rule:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

Plot 1: Learning Curves

Results:

5 Model 3: LSTM

Train an LSTM model for Task A and compare performance.

Tasks

1. Train an LSTM with at least 32 units.
2. Provide validation and test metrics.
3. Extract and plot LSTM gate activations for at least one sequence:

$$f_t, \quad i_t, \quad o_t, \quad \tilde{c}_t, \quad c_t, \quad h_t.$$

4. Interpret the behaviour of forget, input, and output gates.

Gate Activation Plots:

Interpretation:

6 Model 4: GRU

Train a GRU model for Task A and compare with RNN/LSTM.

Tasks

1. Train a GRU with at least 32 units.
2. Extract and plot GRU gates:
$$z_t, \quad r_t, \quad \tilde{h}_t, \quad h_t.$$
3. Compare qualitatively with LSTM gates.

Placeholders

GRU Gate Plots:

7 Comparative Analysis

Provide a summary comparison of the four models:

Model	MAE (Val)	RMSE (Val)	MAE (Test)	RMSE (Test)
FFNN				
Vanilla RNN				
GRU				
LSTM				

Tasks

1. Comment on performance and stability of each model.
2. Discuss which gating mechanisms improved learning and why.

8 CNN Comparison on MNIST and CIFAR-10

In this section, you will compare the behaviour and performance of Convolutional Neural Networks (CNNs) on two widely used image datasets:

- **MNIST:** 70,000 grayscale handwritten digits (simple, low-dimensional).
- **CIFAR-10:** 60,000 color images of 10 object classes (complex, high-dimensional).

This exercise will help you understand how dataset complexity, input dimensionality, and feature structure influence CNN design and performance.

Tasks

1. Train a small CNN on **MNIST** with the following structure (or a comparable one): Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Dense layers. Report training time and accuracy.
2. Train the **same CNN architecture** on **CIFAR-10** without modification. Report training time and accuracy.
3. Analyse and explain:
   - Why does the same CNN achieve significantly different performance on MNIST vs CIFAR-10?
   - How does image dimensionality (1-channel vs 3-channel) change representational difficulty?
   - How does class complexity differ between digits and natural images?
   - What architectural changes would improve CIFAR-10 performance?

Table: CNN Accuracy and Training Time

Dataset	Accuracy (%)	Training Time (mins)
MNIST		
CIFAR-10		

Plots: Learning Curves

Explanation and Comparative Analysis:

9 Student-Captured Image Dataset

Collect your own images (don't use same photos with your classmates) using phone cameras or digital cameras to create a small custom dataset for CNN training. This dataset must be collected ethically under the following conditions:

- Only non-sensitive objects or scenes may be photographed (e.g., books, plants, stationery, vehicles from afar, classroom objects, textures, tables, etc....).
- No identifiable people, faces, license plates, or private information may appear in any image.
- Include a brief dataset description explaining where images were captured, number of samples, and class labels.

Tasks

1. Capture at least 20–30 images in 2–3 classes.
2. Preprocess and label the images appropriately.
3. Train a CNN model on the custom dataset.
4. Compare performance with MNIST, CIFAR-10, and the outsourced dataset, discussing domain shift and dataset quality.